Assignment 4

Exercises:

20.1-1>

Out degree : $O(V+E)$

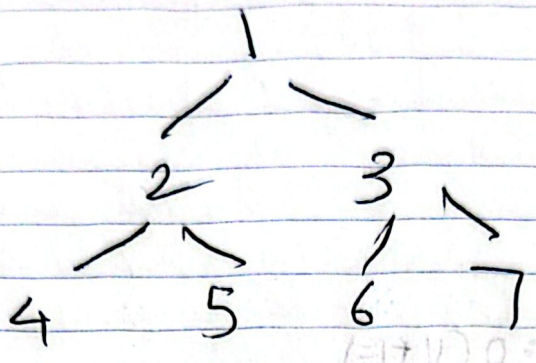
In an adjency list representation, the out-degree of each vertex is simply the length of its adjacency list. We scan through all the V lists, and the total number of entries across all lists is E. So total time is $\Theta(V+E)$

In degree: $O(V+E)$

To compute in degrees, we must scan through every adjacency list and for each edge (u,v), increment the in-degree count of v. This again requires examining all V-lists and all E edges total, giving $\Theta(V+E)$.

Scanned with CamScanner

20.1-2)



This is a complete binary tree on 7 vertices numbered as in a binary heap.

Adjacency list:

1 Adj 2 3 /

2 Adj 1 4 5 /

3 Adj 1 6 7 /

4 Adj 2 /

5 Adj 2 /

6 Adj 3 /

7 Adj 3 /

Space $\Theta(V+E)$

Adjacency matrix:

	1	2	3	4	5	6	7
1	0	1	1	0	0	0	0
2	1	0	0	1	1	0	0
3	1	0	0	0	0	1	1
4	0	1	0	0	0	0	0
5	0	1	0	0	0	0	0
6	0	0	1	0	0	0	0
7	0	0	1	0	0	0	0

Space: $\Theta(V^2)$

The matrix is symmetric since the edges are undirected. The tree has $|V| = 7$ vertices and $|E| = 6$ edges (since a tree on n vertices always has n-1 edges). Each undirected graph appears twice in the adjacency list and corresponds two 1-entries in the matrix.

20.1-3)

Adjacency list:

make
Adj^T with $|V|$ empty lists. Traverse every vertex u in G.V, and for each vertex v in G.adj[u], add u to the list Adj^T[v].

Scanned with CamScanner

Transpose list (G)

1. Create G^T with G.V and empty adjacency list
2. for each u in G.V
3. ~~for each v in G.Adj[u]~~
3. For each V in G.Adj [u]
4. insert u onto G^T.Adj [v]
5. return G^T

Initialization takes $\Theta(V)$. The nested loop iterates over every edge exactly once, taking $\Theta(E)$ total. So the total running time is $\Theta(V+E)$.

Space: $\Theta(V+E)$ for new adjacency list.

Adjacency matrix:

The transpose of the adjacency matrix is simply the matrix transpose: swap rows and columns.

$$a^T_{ij} = a_{ji}$$

Scanned with CamScanner

Transpose matrix: (A)

1. let A^T be a new $|V| \times |V|$ matrix
2. for $i = 1$ to $|V|$
3. for $j = 1$ to $|V|$
4. $A^T[i][j] = A[j][i]$
5. return A^T

We iterate over all V^2 entries of the matrix.

Total running time is $\Theta(V^2)$.

Space: $\Theta(V^2)$ for the new matrix.

20.1-6)

A universal sink has in degree $|V|-1$ and out degree 0.

This means column k of the adjacency matrix is all 1s (except $A[k][k] = 0$) every other vertex points to k.

Row k has is all 0s. k points to no one.

Scanned with CamScanner

20.1-6)
Algorithm:

Start at A[1][1]. Use a pointer (i, j) starting at (1, 1) and walk through the matrix:

```
UNIVERSAL-SINK(A)
  i = 1, j = 1
  while i ≤ |V| and j ≤ |V|
     if A[i][j] = 1
        i = i + 1
     else
        j = j + 1
  if i > |V|
     return "no universal sink"
  for k = 1 to |V|
     if k ≠ i and (A[i][k] = 1 or A[k][i] = 0)
        return "no universal sink"
  return "vertex i is a universal sink"
```

So how this works is if A[i][j] = 1, then vertex i has an outgoing edge, so i cannot be a universal sink. Eliminate i by incrementing i.

From the algorithm:

If A[i][j]=1, then vertex i has an outgoing edge, so i cannot be a universal sink. Eliminate i by incrementing i.

If A[i][j] = 0, then vertex j does not receive an edge from i, so j can not be ~~a~~ universal sink. Eliminate j by incrementing j.

After the while loop, at most one candidate remains (vertex i). We verify it by checking its entire row (all 0s) and column (all 1s except diagonals)

The while loop runs at most 2|V|-1 iterations since each step increments ~~either~~ either i or j, and both are bounded by |V|. The verification step takes O(V).

Total running time is O(V).

20.2-4)

In the standard BFS algorithm using an adjacency list, when we ~~dequeue~~ dequeue a vertex u, we scan $G.Adj[u]$ to find all neighbours. This takes $\Theta(\deg u)$ time per vertex.

With an adjacency matrix, to find all vertices adjacent to u, we must scan the entire row u of the matrix checking all $|V|$ entries. This takes $\Theta(V)$ per vertex.

Since every vertex is enqueued and dequeued at most once, and each dequeue requires scanning a full row of ~~vertices~~ $|V|$ entries, the total time spent exploring neighbours is $\Theta(V \cdot V) = \Theta(V^2)$

Adding $\Theta(V)$ initialization, the total running time is:

$\Theta(V^2)$

This is worse than the adjacency ~~list~~ list version of $\Theta(V+E)$ for sparse graphs (where ~~E~~ E is much less than V^2), but equivalent for dense graphs where $E = \Theta(V^2)$.

Scanned with CamScanner

20.2-8)

The diameter of a tree can be found using two BFS calls.

In a tree, the farthest vertex lies at one end of a diameter path. So we can find the diameter by:

1) Run BFS from any arbitrary vertex s. Let u be the vertex with the largest distance from s.

2) Run BFS from u. Let v be the vertex with the largest distance from u.

3) Diameter is $\delta(u, v)$

The time analysis of the algorithm:

Each BFS runs in $O(V+E)$. Since T is a tree, $|E| = |V| - 1$, so each BFS takes $O(V)$. We run BFS twice, and finding the maximum distance vertex takes $O(V)$ each time.

Total running time is $O(V)$

20.2-8)

Algorithm:

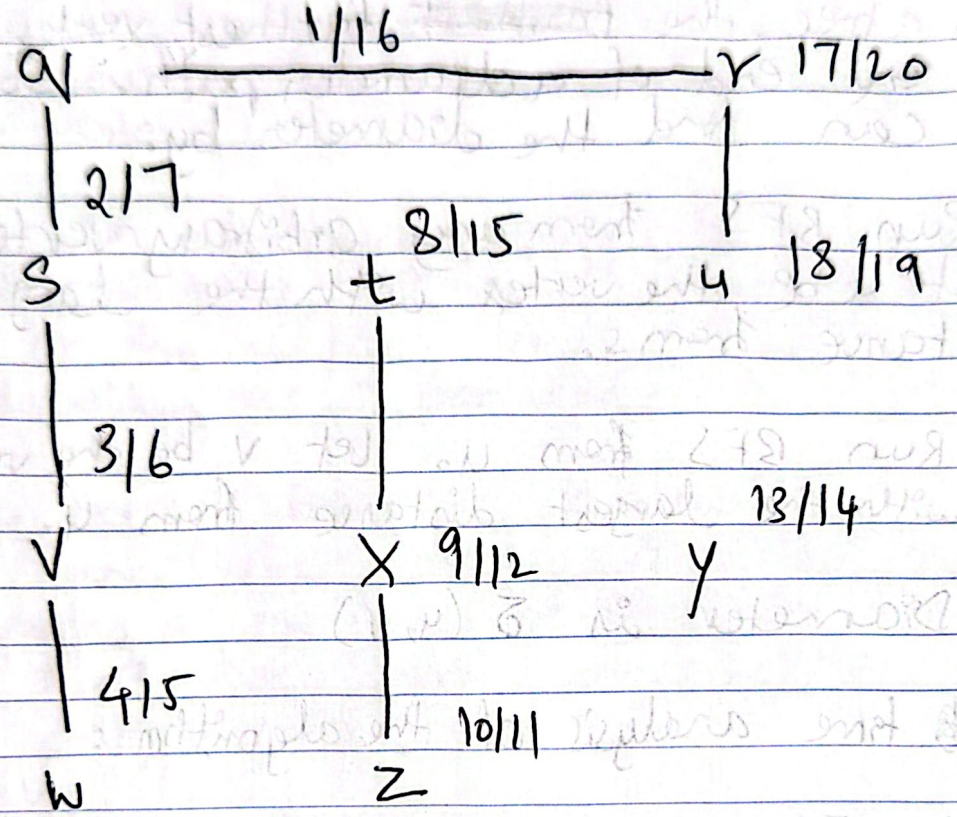
```
TREE-DIAMETER(T)
  pick any vertex s in T.V
  BFS(T, s)
  let u be the vertex with maximum u.d
  BFS(T, u)
  let v be the vertex with maximum v.d
  return v.d
```

Let a, b be a diameter path in T. When we run BFS from an arbitrary vertex s, the farthest vertex u must be one endpoint of some diameter path. This is because in a tree there is exactly one path between any two vertices. If u were not an endpoint of a diameter, then we could find a longer path by extending through the actual diameter endpoint, contradicting that u was farthest from s.

Once we have one endpoint u of a diameter, the second BFS from u finds the farthest vertex v from u, giving us the full diameter path.

20.3-2)

DFS starting at q (alphabetz order).



Discovery / finish time

q : 1/16

s : 2/7

u : 18/19

w : 4/5

y : 13/14

r: 17/20

t: 8/15

v: 3/6

x: 9/12

z: 10/11

Two depth first trees:

→ First tree rooted at:
q: q, s, v, w, t, x, z, y

→ Second tree rooted at:
r: r, u.

Edge classification:

q → s : T

q → t : T

q → w : F

r → u : T

r → y : C

u → y : C

y → q : B

s → v : T

t → x : T

t → y : T

v → w : T

x → z : T

w → s : B

z → x : B

T = Tree
B = Back
F = Forward
C = Cross

Scanned with CamScanner

20.3-7

The Conjecture:

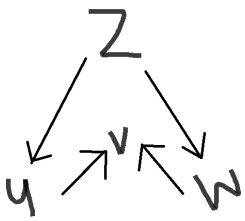
- If a directed graph G has a path from u to v, and in a depth-first search u.d < v.d, then v must be a descendant of u in the DFS forest.
- A counterexample must involve a path where this is not the case, meaning v is discovered via a different path before u's search can reach it.
- This typically involves a cross edge.

Counterexample:

Vertices: {u, v, w, z}

Edges:

- z → u
- z → w
- u → v
- w → v



DFS Execution (alphabetical order):

1. Start at z
    - z.d = 1, mark z gray
    - Explore adjacency list (u, w) alphabetically
2. Visit u
    - u.d = 2, mark u gray
    - Explore (v)
    - Visit v: v.d = 3, mark v gray
    - v has no new neighbors → finish v → v.f = 4
    - Backtrack to u → finish u → u.f = 5

3. Return to z, next neighbor w
    - w.d = 6, mark w gray
    - Explore (v) → v is black (already finished) → cross edge (w, v)
    - Finish w → w.f = 7
4. Finish z → z.f = 8

Vertex: (Discovery, Finish)

z: (1,8)

u: (2,5)

v: (3,4)

w: (6,7)

Here,

- Path u → v exists.
- u.d = 2 < v.d = 3, so conjecture's condition holds.
- But v is not always a descendant of u, because another path (w → v) exists that reuses a finished vertex.
- This makes (w, v) a cross edge, that shows the path u→v has a visited vertex during DFS.

So, even though there is a path from u to v and u.d < v.d, v is not necessarily a descendant of u in the DFS forest. Therefore, this graph is a valid counterexample to the conjecture.

Exercise 20.3-8

The Conjecture:

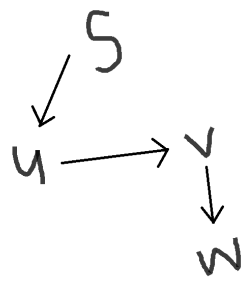
- If a directed graph G contains a path from u to v, then in any DFS of G, we must have v.d ≤ u.f.
- So basically, if there's a path from u to v, then DFS will discover v before u finishes.

When DFS discovers u:

- If there's a path from u to v, DFS will *eventually* reach v unless v was already discovered.
- Depending on v's color when u explores it:
    - White: v will be discovered during u's DFS
    - Gray: v is an ancestor
    - Black: v already finished

There's no way for v to be discovered after u finishes (which would require u.f < v.d), because if a path u→v exists, DFS from u would have reached v while u was still gray.

Example demonstrating this:



Vertices: {s, u, v, w}

Edges:

- s → u
- u → v
- v → w

DFS Execution (alphabetical order):

Start at s:

- s.d = 1
- Visit u → u.d = 2

- Visit v → v.d = 3
- Visit w → w.d = 4
- w has no outgoing edges → finish w, w.f = 5
- Backtrack to v → finish, v.f = 6
- Backtrack to u → finish, u.f = 7
- Backtrack to s → finish, s.f = 8

Vertex: (Discovery, Finish)

s: (1,8)

u: (2,7)

v: (3,6)

w: (4,5)

Reason:

- When u is discovered, v is still white.
- DFS immediately explores v, making it a descendant.
- Therefore u.d < v.d < v.f < u.f.

In every case, v.d < u.f. The only way u.f < v.d could happen is if v were discovered after u finished, but DFS can't do that if a path u → v exists.

Therefore, the conjecture is true.